CUADERNO DE PRÁCTICAS // DESARROLLO WEB

D3 · UD2
SPRING BOOT

FORMULARIOS ENCADENADOS

Kerin Aguilera

JAVA 21 · SPRING BOOT · OCTUBRE 2026

1. Objetivo de la práctica
2. Recorrido del formulario
3. Primera versión: formularios encadenados
4. Primera lógica del controlador
5. Segunda versión: navegación por pasos
6. Resultado, navegación y conservación de datos
7. Resumen de conceptos

ENTRADA SEC. // # FORMULARIOS ENCADENADOS CON SPRING BOOT

Objetivo

En esta práctica se construye un formulario dividido en varios pasos. En cada paso se solicitan datos distintos y, al final, se muestra un resumen con toda la información introducida.

El ejemplo utiliza Spring Boot, Thymeleaf y un objeto Java (`TusDatos`) para enlazar los campos del formulario con el modelo. Como las peticiones HTTP no conservan automáticamente el estado entre una página y otra, los datos ya introducidos deben volver a enviarse en los formularios siguientes.

Recorrido de la práctica

1. Se solicita el nombre.
2. Se seleccionan las aficiones.
3. Se elige un lugar.
4. Se indica el sexo.
5. Se muestra un resumen con los datos y las opciones para volver atrás o empezar de nuevo.

Primera versión: avanzar entre formularios

Paso 1 · Introducir el nombre

El primer formulario no necesita campos ocultos: todavía no hay datos anteriores que conservar. El atributo `th:object` vincula los campos al objeto que recibe la plantilla.

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Formulario 1</title>
</head>
<body>
    <form method="post" th:action="@{/procesar}" th:object="${tusDatos}">
        <label for="nombre">Nombre:</label>
        <input id="nombre" type="text" th:field="*{nombre}">

        <button type="submit" name="contador" value="1">Enviar</button>
    </form>
</body>
</html>
```

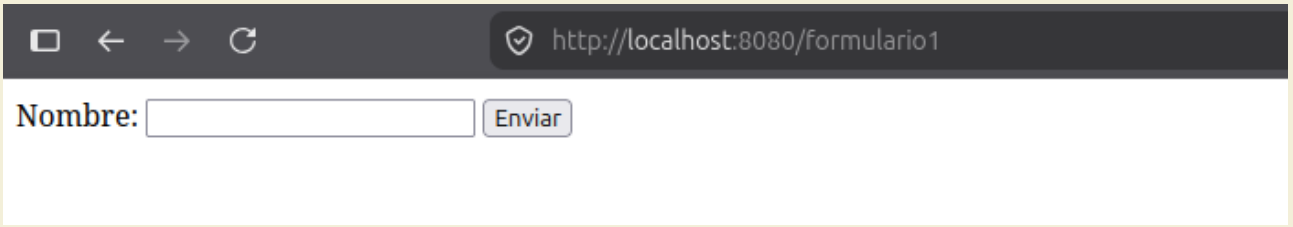
Para comprobar los datos recibidos, se pueden imprimir temporalmente en la consola desde el controlador:

```
System.out.println(datos);
System.out.println(siguiente);
```

Por ejemplo, después de enviar el nombre, la consola puede mostrar:

```
TusDatos(nombre=Ruvik, Aficciones=[], lugar=null, datosFinales=[])
1
```

Así se ve el primer formulario:



Paso 2 · Seleccionar las aficiones

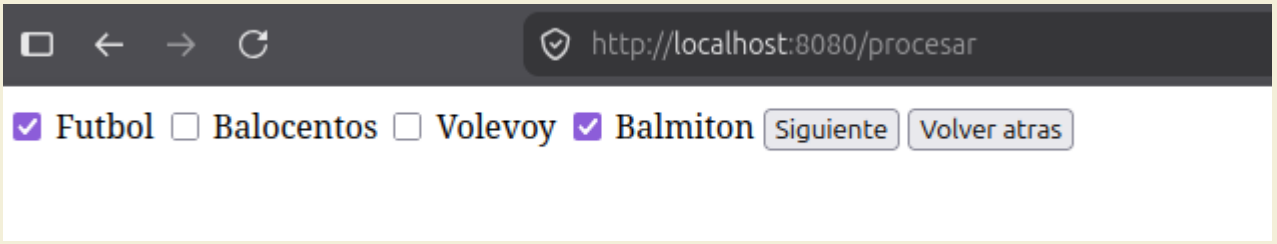
El segundo formulario recibe el nombre del paso anterior. El campo oculto `nombre` permite volver a enviarlo junto con las nuevas selecciones.

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Formulario 2</title>
</head>
<body>
    <form method="post" th:action="@{/procesar}" th:object="${datos}">
        <fieldset>
            <legend>Selecciona tus aficiones:</legend>

            <label><input type="checkbox" th:field="*{Aficciones}"
value="Fútbol"> Fútbol</label>
            <label><input type="checkbox" th:field="*{Aficciones}"
value="Baloncesto"> Baloncesto</label>
            <label><input type="checkbox" th:field="*{Aficciones}"
value="Voleibol"> Voleibol</label>
            <label><input type="checkbox" th:field="*{Aficciones}"
value="Bádminton"> Bádminton</label>
        </fieldset>

        <button type="submit" name="contador" value="1">Enviar</button>
        <input type="hidden" th:field="*{nombre}">
    </form>
</body>
</html>
```

Vista del formulario:



En la primera implementación, la consola muestra cómo se incorporan los datos paso a paso:

```
TusDatos(nombre=Ruvik, Aficciones=[], lugar=null, datosFinales=[])
1
TusDatos(nombre=Ruvik, Aficciones=[Baloncesto, Bádminton], lugar=null,
datosFinales=[])
2
```

Paso 3 · Elegir un lugar

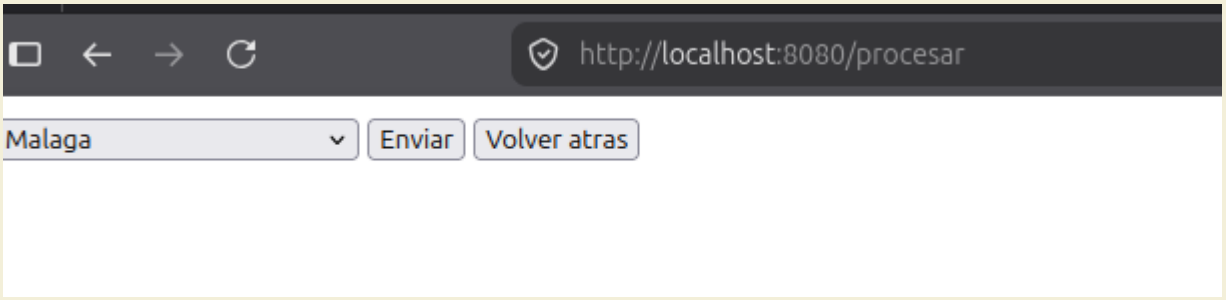
El tercer formulario añade el lugar y vuelve a enviar los datos de los pasos anteriores. Para conservar una colección, como `Aficciones`, se genera un campo oculto por cada elemento.

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Formulario 3</title>
</head>
<body>
    <form method="post" th:action="@{/procesar}" th:object="${datos}">
        <label for="lugar">Lugar:</label>
        <select id="lugar" th:field="*{lugar}">
            <option value="">-- Selecciona una opción --</option>
            <option value="Madrid">Madrid</option>
            <option value="Sevilla">Sevilla</option>
            <option value="Málaga">Málaga</option>
        </select>

        <button type="submit" name="contador" value="1">Enviar</button>

        <input type="hidden" th:field="*{nombre}">
        <th:block th:each="aficion : ${datos.Aficciones}">
            <input type="hidden" name="Aficciones" th:value="${aficion}">
        </th:block>
    </form>
</body>
</html>
```

Vista del formulario:



Después de seleccionar, por ejemplo, Sevilla, el objeto contiene los datos de los tres pasos:

```
TusDatos(nombre=Ruvik, Aficciones=[], lugar=null, datosFinales=[])
1
TusDatos(nombre=Ruvik, Aficciones=[Baloncesto, Bádminton], lugar=null,
datosFinales=[])
2
TusDatos(nombre=Ruvik, Aficciones=[Baloncesto, Bádminton], lugar=Sevilla,
datosFinales=[])
3
```

▶ Primera lógica del controlador

En la primera versión, un método `@PostMapping` procesa todos los envíos. El parámetro `contador` indica qué paso se ha completado y determina qué plantilla se muestra a continuación.

```
@PostMapping("/procesar")
public String procesarDatos(
        @ModelAttribute TusDatos datos,
        @RequestParam("contador") String contador,
        Model modelo) {

    siguiente += Integer.parseInt(contador);
    System.out.println(datos);
    System.out.println(siguiente);

    if (siguiente == 1) {
        modelo.addAttribute("datos", datos);
        return "Formulario2";
    }

    if (siguiente == 2) {
        modelo.addAttribute("datos", datos);
        return "Formulario3";
    }

    modelo.addAttribute("datos", datos);
    return "Resultado";
}
```

Esta primera aproximación utiliza un contador global:

```
static int siguiente = 0;
```

Es una solución sencilla para probar el recorrido, pero no es adecuada para varios usuarios: el valor se comparte entre todas las peticiones y puede quedar desincronizado. La siguiente versión obtiene el paso actual de cada petición, en lugar de guardarlo en una variable estática.

Modelo de datos

`TusDatos` reúne la información que se va completando en los distintos formularios. Lombok genera los métodos habituales, como los *getters* y *setters*, mediante `@Data`.

```
package ruvik.d3ud2;

import lombok.Data;
import java.util.ArrayList;

@Data
public class TusDatos {
    private String nombre;
    private ArrayList<String> Aficciones = new ArrayList<>();
    private String lugar;
    private ArrayList<String> datosFinales = new ArrayList<>();
    private String sexo;
}
```

▶ Segunda versión: navegación con el valor de `contador`

La nueva versión utiliza el valor recibido en la petición para decidir qué vista mostrar. Así se evita acumular el estado en una variable global.

```
package ruvik.d3ud2.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import ruvik.d3ud2.TusDatos;

@Controller
public class ControladorFormulariosEncadenado {

    @GetMapping("/formulario1")
    public String formulario1(@ModelAttribute TusDatos tusDatos, Model
model) {
        model.addAttribute("tusDatos", tusDatos);
        return "Formulario1";
    }

    @PostMapping("/procesar")
    public String procesarDatos(
            @ModelAttribute TusDatos datos,
            @RequestParam("contador") Integer contador,
            Model modelo) {
```

```
        System.out.println(datos);
        System.out.println(contador);

        if (contador == 0) {
            return "redirect:/formulario1";
        }

        modelo.addAttribute("datos", datos);
        modelo.addAttribute("contador", contador);

        if (contador == 1) {
            return "Formulario2";
        }
        if (contador == 2) {
            return "Formulario3";
        }
        if (contador == 3) {
            return "Formulario4";
        }

        return "Resultado";
    }
}
```

Los botones envían el número del paso siguiente. En los botones para retroceder, envían el número del paso anterior. El botón para empezar de nuevo envía `0`.

Paso 4 · Añadir el sexo

El cuarto formulario incorpora una selección con botones de opción. También debe reenviar los valores anteriores para que sigan disponibles al procesar el formulario.

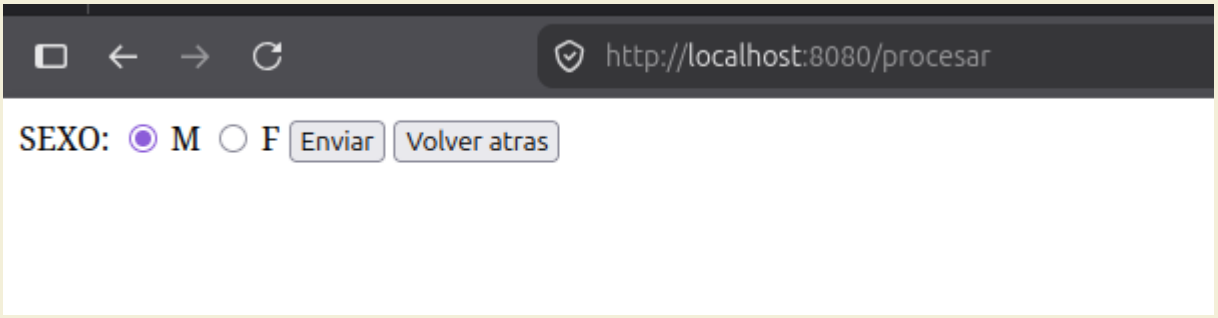
```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Formulario 4</title>
</head>
<body>
    <form method="post" th:action="@{/procesar}" th:object="${datos}">
        <fieldset>
            <legend>Sexo:</legend>
            <label><input type="radio" th:field="*{sexo}"
value="M"> M</label>
            <label><input type="radio" th:field="*{sexo}"
value="F"> F</label>
        </fieldset>

        <button type="submit" name="contador" th:value="${contador
+ 1}">Enviar</button>
```

```
        <button type="submit" name="contador" th:value="${contador -
1}">Volver atrás</button>

        <input type="hidden" th:field="*{nombre}">
        <th:block th:each="aficcion : ${datos.Aficciones}">
            <input type="hidden" name="Aficciones" th:value="${aficcion}">
        </th:block>
        <input type="hidden" th:field="*{lugar}">
    </form>
</body>
</html>
```

El formulario se muestra así:



▶ Resultado y navegación

La vista final presenta los datos recibidos. `th:text` inserta valores en el contenido HTML y `th:each` recorre la lista de aficiones.

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Resumen de datos</title>
</head>
<body>
    <h1 th:text="|Te llamas: ${datos.nombre}|"></h1>

    <p>Tus aficiones:</p>
    <ul>
        <li th:each="aficcion : ${datos.Aficciones}"
th:text="${aficcion}"></li>
    </ul>

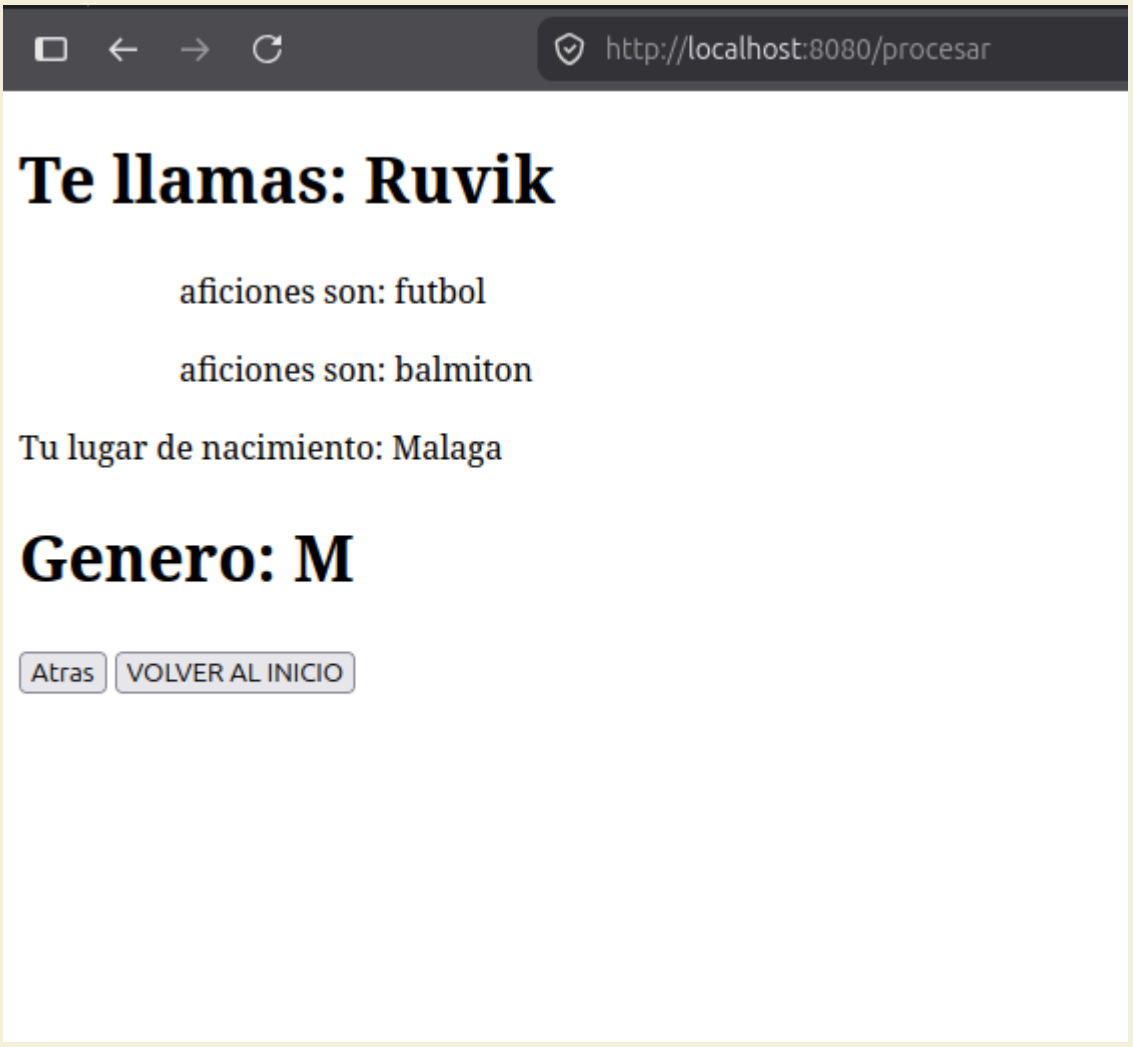
    <p th:text="|Tu lugar: ${datos.lugar}|"></p>
    <p th:text="|Sexo: ${datos.sexo}|"></p>

    <form method="post" th:action="@{/procesar}" th:object="${datos}">
```

```
        <button type="submit" name="contador" th:value="${contador -
1}">Volver atrás</button>
        <button type="submit" name="contador" value="0">Volver
al inicio</button>

        <input type="hidden" th:field="*{nombre}">
        <th:block th:each="aficcion : ${datos.Aficciones}">
            <input type="hidden" name="Aficciones" th:value="${aficcion}">
        </th:block>
        <input type="hidden" th:field="*{lugar}">
        <input type="hidden" th:field="*{sexo}">
    </form>
</body>
</html>
```

La captura muestra el resultado final:



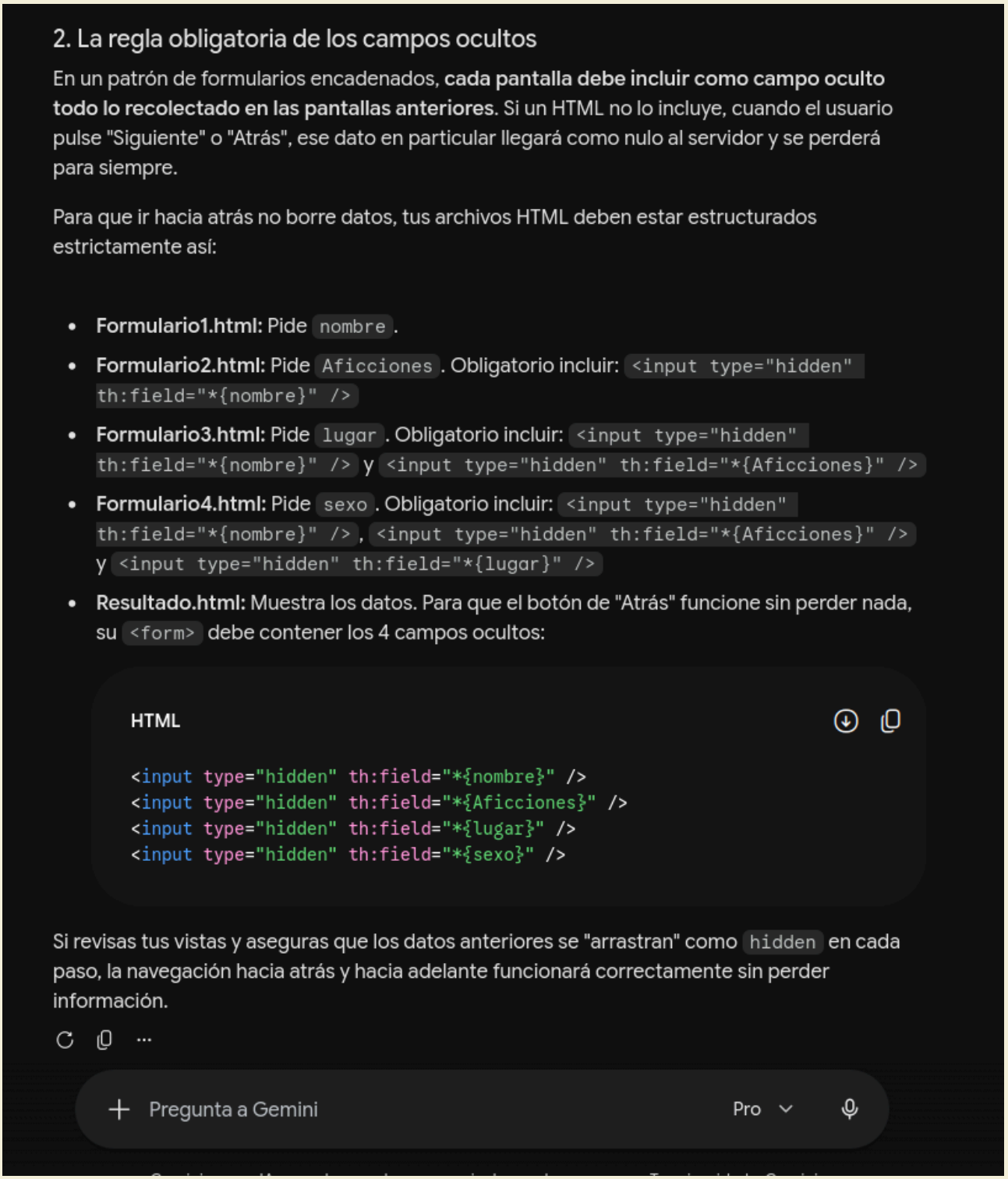
Problema · Se pierden los datos al volver atrás

Al probar la navegación hacia atrás, los datos de los pasos anteriores desaparecían. La causa era que el formulario enviaba solo el campo visible del paso actual. Los valores previos no llegaban al controlador y, por tanto, no podían volver a incluirse en el modelo.

Solución · Reenviar los valores anteriores

Se añadieron campos ocultos a los formularios para volver a enviar los datos ya introducidos. Para campos simples se puede utilizar `th:field`; para la lista de aficiones se genera un campo oculto por cada elemento.

Los campos ocultos sirven para transportar valores entre peticiones, pero el usuario puede modificarlos desde el navegador. No deben utilizarse para guardar información sensible ni para confiar en valores que el servidor no haya validado.



▸ Resumen de conceptos

- `@ModelAttribute` enlaza los campos enviados con el objeto `TusDatos`.
- `th:object` indica el objeto del formulario y `th:field` vincula sus propiedades a los controles.
- Los campos ocultos reenvían datos previos que no aparecen como controles editables en el paso actual.
- El parámetro `contador` permite avanzar, retroceder o volver al inicio.
- Un contador `static` comparte estado entre usuarios; es preferible obtener el paso de cada petición o gestionar el estado con una solución apropiada para la aplicación.

▶ Recursos

- **Repositorio del proyecto en GitHub**
- Stack Overflow
- Gemini

FIN DE LA TRANSMISIÓN // D3 · UD2